

TzakGPT

A Local CLI AI Agent

Technical White Paper

Version 1.0 | June 2026

Powered by DeepSeek | Local-First Architecture

TzakGPT is a terminal-based AI coding assistant that operates locally on the user's machine. It combines streaming large-language-model inference with agentic tool execution, file manipulation, shell command orchestration, and persistent session memory. This white paper documents its architecture, design philosophy, complexity characteristics, and the engineering trade-offs that make it simultaneously powerful and lightweight.

All code examples herein reflect the TzakGPT v1.0 codebase.

Table of Contents

1. Executive Summary	3
2. Architecture Overview	4
3. The Core Engine -- main.py	6
4. AI Integration Layer -- clients.py	8
5. The Soul -- soul.py	10
6. Tool System -- tools.py	12
7. Session Management -- session.py	13
8. Display & UX -- display.py	14
9. Slash Commands & Interactivity	16
10. Context Management & The Sliding Window	18
11. Token Economy & Efficiency	20
12. Streaming Architecture	22
13. Safety & Security Model	23
14. Project Structure & Dependencies	24
15. Performance Characteristics	25
16. The System Prompt: Voice & Persona	26
17. Future Roadmap	27
18. Conclusion	28

Note: All page references in this table are approximate.

1. Executive Summary

TzakGPT is a local, terminal-based AI coding assistant that operates as a command-line interface (CLI) application running directly on the user's Windows, macOS, or Linux machine. It is not a cloud service, not a browser extension, and not an IDE plugin. It is a standalone Python program that sits in the terminal and acts as a capable engineering colleague -- reading files, writing code, running shell commands, and maintaining conversational context across sessions.

The system is built on top of DeepSeek's large language models (both the fast 'Flash' variant and the more capable 'Pro' variant) accessed via an OpenAI-compatible API. It streams responses token-by-token into a live-updating Rich panel with full Markdown rendering. It is agentic: the model can autonomously decide to invoke tools (file I/O, directory listing, shell execution) and TzakGPT will execute them, feed results back to the model, and continue the loop until the task is complete.

Key differentiators include: (a) a sophisticated sliding-window context management algorithm that prevents token bloat without losing semantic coherence; (b) a colour-coded token usage bar with context-fill warnings; (c) persistent JSON-based session saves with metadata previews; (d) a non-blocking interrupt mechanism (press X to stop the model mid-thought); and (e) a carefully crafted system prompt that gives TzakGPT a distinct, warm-yet-competent personality.

The entire codebase is approximately 800 lines of Python spread across six modules, each with a single responsibility. The architecture is intentionally flat -- there are no class hierarchies, no abstract base classes, and no dependency injection frameworks. This is engineering pragmatism: the system is small enough that functions and module-level state suffice.

2. Architecture Overview

TzakGPT follows a modular, single-process architecture where each source file encapsulates one coherent subsystem. The modules communicate through function calls and shared module-level state (particularly in `session.py`). There is no message bus, no event system, and no asynchronous I/O. The system is synchronous from top to bottom, with streaming handled by a Python generator in `clients.py` that yields (`event_type`, `data`) tuples consumed by the agent loop in `main.py`.

2.1 Module Map

The six source modules and their responsibilities:

- `main.py` -- Entry point. UI loop (`prompt_toolkit`), agent orchestration, streaming display via Rich Live panels, slash command handling, model selection, session save/load UI, token bar rendering.
- `clients.py` -- DeepSeek API integration. Wraps the OpenAI Python client. Provides both streaming (`ask_deepseek_stream`) and non-streaming (`ask_deepseek`) calls. Handles tool-call delta accumulation across streaming chunks.
- `soul.py` -- The 'soul' of the assistant. Contains the system prompt, tool definitions (JSON Schema), the sliding-window context summarisation algorithm, a greeting pool, and the `build_payload()` function that assembles the full message array sent to the API.
- `tools.py` -- Local tool implementations. File reading, file writing (with unified diff generation via `difflib`), shell command execution (via `subprocess` with a 60-second timeout), and directory listing.
- `session.py` -- Telemetry and persistence. Tracks token counts, action logs, and turn counts as module-level state. Provides serialisation for `/save` and deserialisation for `/load`.
- `display.py` -- TUI rendering utilities. Colour-coded diff panels, action status labels, shell command confirmation prompts (Y/N/E), and spinner management.

2.2 Data Flow

The data flow through the system follows a clean cycle: (1) User input is collected via `prompt_toolkit` with slash-command auto-completion. (2) Slash commands are intercepted and handled locally; normal messages are appended to `conversation_history`. (3) `soul.build_payload()` assembles the full message array -- system prompt with working directory context and session summary, plus the sliding-window-filtered conversation history. (4) `main.agent_loop()` calls `clients.ask_deepseek_stream()` and renders tokens live via Rich. (5) If the model returns tool calls, `main.py` executes them via `tools.py`, feeds results back as 'tool' role messages, and loops. (6) On final text response, the conversation is appended and the token bar is updated.

3. The Core Engine -- `main.py`

`main.py` is the largest module (~400 lines) and serves as the application's backbone. It contains the `main()` entry point, the `agent_loop()` streaming orchestrator, the slash command handler, the token bar renderer, session save/load UI, and the model chooser.

3.1 The Agent Loop

The `agent_loop()` function is the heart of TzakGPT's agentic behaviour. It implements a bounded iteration loop (max 10 iterations) that streams responses from DeepSeek and executes tool calls as they arrive. Key design elements:

- Streaming with interruption: Before each API call and between each chunk, `check_stop_key()` polls `msvcrt.kbhit()` (Windows) for the 'x' key. If detected, the loop returns immediately with an '[Interrupted by user]' placeholder. This is non-blocking and does not require threading.
- Tool execution with visual feedback: File operations show a spinner with cycling status text ('Thinking...', 'Reading your files...', etc.). Shell commands require Y/N/E confirmation before execution. After any tool takes >2 seconds, a duration notice is printed and the terminal bell rings (if enabled).
- Token tracking per turn: Each API call's input and output token counts are accumulated via `session.add_tokens()`. Turn-level totals are returned to the caller for the token bar display.
- Error resilience: If the stream yields an 'error' event, the loop prints the error in red and returns None, which causes `main()` to pop the last user message and continue without crashing.
- Transient streaming: The Rich Live panel uses `transient=True`, meaning the in-progress streaming panel disappears when the loop exits. Only the final polished Markdown panel (rendered in `main()`) remains visible. This prevents visual clutter while maintaining the sense of live streaming.

3.2 Slash Command System

Slash commands (`/`, `/help`, `/clear`, `/status`, `/tokens`, `/save`, `/load`, `/model`, `/bell`) are handled entirely client-side by `handle_slash_command()`. Commands that are not recognised fall through to the AI as normal messages. The `SlashCompleter` class (implementing `prompt_toolkit`'s Completer protocol) only activates when the input starts with `/`, showing a dropdown with command descriptions.

Notable slash command behaviours: `/clear` asks for Y/N confirmation before wiping history, then optionally offers to delete the most recent saved session file. `/model` accepts 'pro' or 'flash' as arguments or prompts interactively with F/P key detection. `/load` shows a numbered list of recent sessions with metadata (turn count, model used, relative timestamp) and accepts numeric selection via `readchar`.

3.3 Token Bar

The `show_token_line()` function renders a 10-character Unicode block bar (filled + empty blocks) showing context utilisation as a percentage of the 600,000 token maximum. The bar is colour-coded: grey below 10%, green below 30%, yellow below 60%, orange below 85%, and bold bright red above 85%. Below 10% utilisation with fewer than 3 turns, the bar is hidden entirely to reduce noise. A warning message appears above 300,000 tokens (50%) suggesting `/clear` or `/save`.

4. AI Integration Layer -- `clients.py`

clients.py is the sole interface between TzakGPT and the DeepSeek API. It uses the official OpenAI Python client library pointed at DeepSeek's API endpoint (api.deepseek.com). This is possible because DeepSeek implements the OpenAI chat completions API contract.

4.1 Model Selection

The module maintains a module-level `_model` variable (default: 'deepseek-v4-flash') that can be changed at runtime via `set_model()`. Two helper functions, `get_model()` and `get_model_display()`, return the full model key and a user-friendly short name ('Pro' or 'Flash') respectively. This design allows model switching without restarting the application -- the next API call simply uses the new model.

4.2 Non-Streaming Calls

`ask_deepseek()` is used for internal tasks such as the sliding-window summary generation. It makes a synchronous `chat.completions.create()` call and returns a tuple of (kind, message, usage_dict). The kind is either 'text' for a normal response or 'tool_call' if the model requested tool invocation. Token usage is extracted from `completion.usage`.

4.3 Streaming Calls

`ask_deepseek_stream()` is a Python generator that yields (event_type, data) tuples. It is used for all user-facing responses. The generator handles several complexities:

- Delta accumulation: DeepSeek streams tool calls across multiple chunks (index, id, function.name, function.arguments arrive separately). The generator accumulates these into a `collected_tool_calls` list, building up each tool call's structure incrementally.
- Usage extraction: Token usage typically arrives in the final chunk. The generator checks `chunk.usage` on each iteration and updates the usage dict when present.
- Event types: The generator yields four event types: 'text' for incremental content to display, 'tool_calls' when the model finishes and tool calls are pending (includes accumulated tool calls and usage), 'done' for a final text response (includes full text and usage), and 'error' for any exception.
- Error handling: Both API-level exceptions and stream iteration exceptions are caught and yielded as 'error' events rather than propagating. This lets the agent loop handle errors gracefully.

5. The Soul -- soul.py

soul.py is where TzakGPT's identity, behaviour, and context management live. It contains the system prompt, tool definitions, greeting pool, and the sliding-window context algorithm.

5.1 System Prompt Design

The `SYSTEM_PROMPT` constant (~800 characters) defines TzakGPT's entire persona and behavioural constraints. It is structured in three sections: TONE AND VOICE (personality directive), tool availability and usage rules, and NARRATION AND SELF-VERIFICATION RULES (operational constraints). This prompt is the single most important string in the codebase -- it is what transforms a generic LLM into TzakGPT.

The prompt's tone instructions are notably specific: 'direct, warm, and alert,' 'concise when they move fast, more detailed when they slow down,' 'acknowledge frustration before offering solutions,' 'express mild satisfaction without cheerleading,' 'never use emoji.' These constraints produce a consistent, professional personality that feels like a skilled colleague rather than a corporate chatbot.

The operational rules encode several safety and quality guarantees: always read before writing, always verify after writing, never run shell commands without instruction, stop on tool errors rather than continuing silently, and match the user's energy level. These rules are not aspirational -- the model follows them because they are structured as direct imperatives in the system prompt.

5.2 Tool Definitions

TOOLS_DEFINITION is a list of four JSON Schema objects following OpenAI's function-calling format. Each tool defines a name, description, and parameters schema: `read_file` (path required), `write_file` (path and content required), `run_command` (cmd required), and `list_directory` (path optional). The schemas are passed to the API with `tool_choice='auto'`, allowing the model to decide when to invoke each tool.

5.3 build_payload()

This function assembles the complete message array for each API call. It dynamically injects the current working directory, file listing, and subdirectory listing into the system message. It also appends a session activity summary from `session.py` and applies the sliding-window transformation. The result is a context-rich prompt that gives the model awareness of the user's filesystem without requiring the user to repeatedly describe their environment.

5.4 Greeting Pool

GREETINGS is a list of eight short phrases ('Ready.', 'What are we building?', 'Clear terminal, clear mind...', etc.) from which one is randomly selected at startup. This tiny detail contributes significantly to the perception of personality -- each session feels slightly different.

6. Tool System -- tools.py

`tools.py` implements the four local tools that TzakGPT can invoke. Each function is pure I/O with explicit error handling wrapped in `try/except` blocks.

6.1 read_file()

Reads a file's entire contents with UTF-8 encoding. On any exception, returns an 'ERROR: ...' string rather than raising. This error string convention is critical: the model sees the error in its tool result and can adjust its approach, rather than the application crashing.

6.2 write_file()

Writes new content to a file. If the file already exists, reads the old content first. Uses Python's `difflib.unified_diff()` to generate a standard unified diff between old and new content. The diff is returned as a list of strings, which `display.py` renders as a colour-coded panel. This gives the user immediate visual confirmation of every change.

6.3 run_command()

Executes a shell command via `subprocess.run()` with `shell=True`, capturing both `stdout` and `stderr`. A 60-second timeout prevents runaway processes. On timeout, returns a specific error message. On any other exception, returns a generic error. The combined output is stripped; empty output returns '(no output)' for clarity.

6.4 list_directory()

Lists files and subdirectories at a given path (defaults to current directory). Returns a dict with 'files' and 'dirs' keys, each containing alphabetically sorted lists. On error, returns empty lists with an 'error' key.

7. Session Management -- session.py

`session.py` maintains module-level state for token tracking, action logging, and turn counting. This design choice -- module-level globals rather than a class instance -- keeps the API simple and avoids passing a session object through every function signature.

The `record()` function appends timestamped entries to the `_log` list with action name, detail string, and error flag. `get_log_as_text()` formats the log as a human-readable table (used by `/status`). `get_summary_for_prompt()`

produces a compressed single-line summary of actions, which gets injected into the system prompt so the model knows what has already happened in the session.

Token tracking uses `add_tokens(input, output)` which increments `_input_tokens`, `_output_tokens`, and `_turns`. `get_token_totals()` returns a dict with `turns`, `input`, `output`, and `total`. The restore functions (`restore_log`, `restore_tokens`) are used during session loading to repopulate state from saved JSON.

8. Display & UX -- `display.py`

`display.py` contains the Rich-based TUI rendering components that give TzakGPT its polished terminal appearance.

8.1 Visual Diffs

`show_diff()` parses unified diff output and renders it as a Rich Panel with colour-coded lines: red for removals (prefixed with -), green for additions (+). Line numbers are shown in the margin. The panel width adapts to the terminal size (via `shutil.get_terminal_size()`) but is capped at 120 columns. Border colour varies by action type: green for writes, blue for reads, yellow for shell commands.

8.2 Action Labels

`show_action()` prints a styled label ('Reading', 'Writing', 'Listing', 'Running') with an optional detail string. The colour coding is consistent: bright blue for reading, bright yellow for writing, cyan for listing, yellow for running.

8.3 Command Confirmation

`confirm_command()` displays the proposed shell command and waits for a single keypress: Y to run, N to skip, E to edit. Editing returns a tuple ('e', `edited_command`) so the caller can re-prompt with the modified command. This three-way confirmation is a critical safety feature -- it prevents accidental execution of destructive commands while still allowing the user to refine the model's suggestion.

8.4 Spinner Text Pool

A list of five spinner texts ('Thinking...', 'Reading your files...', 'Consulting DeepSeek...', 'Almost done...', 'Working on it...') is cycled during tool execution and the initial thinking phase. The `_CyclingSpinner` class in `main.py` advances the text every ~40 render frames (~2.7 seconds at 15 fps), creating a sense of activity without repetitive text.

9. Slash Commands & Interactivity

TzakGPT implements nine slash commands that provide session control without leaving the chat interface. The commands are managed through a single source-of-truth `SLASH_COMMANDS` dict, which both the `SlashCompleter` and the `/help` display reference.

<code>/help</code>	Displays all slash commands in a Rich table
<code>/clear</code>	Resets conversation history with Y/N confirmation; optionally deletes latest saved session file
<code>/status</code>	Shows timestamped session activity log (every file read, write, command run)
<code>/tokens</code>	Displays detailed token statistics: turns, input, output, total, context usage percentage with colour-coded bar
<code>/save [name]</code>	Saves session as JSON in <code>src/sessions/</code> with auto-increment naming (<code>session_001.json</code>). Collision-safe with
<code>/load [name]</code>	Restores a saved session; warns if current history will be replaced. Without argument, shows interactive picker
<code>/model [pro flash]</code>	Switches the AI model; without argument, prompts interactively with F/P key detection
<code>/bell</code>	Toggles terminal bell on response completion on/off

The interactive session picker (shown at startup and via `/load`) displays up to 10 saved sessions with metadata parsed without loading full history: turn count, model used, and relative timestamp ('3h ago', 'just now'). This lightweight metadata extraction avoids the performance cost of deserialising large conversation histories just for display.

10. Context Management & The Sliding Window

Long conversations with an LLM inevitably hit context limits. TzakGPT implements a sliding-window algorithm in `soul._apply_sliding_window()` that addresses this problem with a novel approach: dynamic keep-window sizing with AI-powered summarisation.

10.1 Algorithm

The algorithm counts user turns (messages with `role='user'`). For N user turns where $N > 6$: the keep-from point is $\text{floor}(N/2) + 1$. This means as the conversation grows, proportionally more turns are collapsed. At 7 turns, turns 1-3 are collapsed (keeping 4-7). At 20 turns, turns 1-10 are collapsed (keeping 11-20). At 100 turns, turns 1-50 are collapsed (keeping 51-100).

The collapsing process uses the non-streaming `ask_deepseek()` to generate a one-paragraph summary of the older messages. This summary is prepended as a system message: 'Summary of earlier conversation: [summary text]'. The recent messages are kept verbatim.

This approach has several advantages over naive truncation: (a) semantic continuity is preserved through the summary; (b) recent context (which is most relevant) remains 100% intact; (c) the summarisation call is itself inexpensive (one API call every several turns); and (d) the threshold of 6 turns means short conversations never trigger the mechanism.

10.2 Token Budget

TzakGPT defines two token thresholds: `CONTEXT_WARNING_THRESHOLD` at 300,000 tokens (50% of the 600,000 maximum) and `MAX_CONTEXT_TOKENS` at 600,000. The warning threshold triggers a yellow advisory suggesting `/clear` or `/save`. The maximum is used for the progress bar's denominator. DeepSeek's actual context window is significantly larger than 600K, but this conservative threshold keeps responses fast and costs predictable.

10.3 Dynamic Context Injection

Beyond the sliding window, `build_payload()` injects live context into every system message: the current working directory, the list of files and subdirectories, and a summary of all actions taken in the current session. This gives the model filesystem awareness without requiring the user to describe their setup in every message.

11. Token Economy & Efficiency

Token consumption is the primary cost driver for any LLM-powered application. TzakGPT employs several strategies to optimise token usage without sacrificing quality.

11.1 Tracking Infrastructure

`session.py` tracks input and output tokens separately, accumulated across all turns. The `get_token_totals()` function returns a dict with turns, input, output, and total. This data is displayed via `/tokens` and the per-response token bar, giving the user constant visibility into consumption.

11.2 Efficiency Strategies

- Sliding window summarisation (Section 10): Collapses old turns into a single summary paragraph, converting potentially thousands of tokens of conversation into ~100-200 tokens of summary.
- Tool result verbosity control: File read results are returned as-is (the model needs full content), but directory listings use a compact JSON format and command outputs are trimmed.
- System prompt caching: The system prompt is a ~800 character constant; it does not grow with the conversation. Only the dynamic context (working directory, session summary) varies.
- Model choice: Flash model (~1/3 the cost of Pro) is the default. Users switch to Pro only when they need

deeper reasoning.

- Token bar visibility: Below 10% utilisation with fewer than 3 turns, the token bar is hidden -- a small UX optimisation that also subtly discourages premature context anxiety.

11.3 Cost Characteristics

A typical coding session of 20 turns with Flash consumes approximately 150,000-300,000 total tokens (input + output). At DeepSeek's published rates, this is sub-\$0.01 USD per session. Pro model sessions at the same length would be roughly 3x the cost but provide noticeably better reasoning for complex refactoring tasks.

12. Streaming Architecture

TzakGPT's streaming architecture is built on Python generators and Rich's Live display system. The entire pipeline is synchronous -- no async, no threads, no callbacks.

12.1 Generator Pipeline

`clients.ask_deepseek_stream()` is a generator that yields (event, data) tuples. The agent loop iterates over this generator inside a Rich Live context manager. Each 'text' event appends to a collected_text string and updates the Live panel with rendered Markdown. This creates the illusion of real-time typing, with Markdown formatting applied incrementally.

The Live panel is configured with `refresh_per_second=15` and `transient=True`. The transient flag means the streaming panel is removed from the terminal when the Live context exits -- it is replaced by the final polished Panel rendered in `main()`. This avoids the common 'ghost panel' problem where streaming UIs leave stale frames in the scrollbar.

12.2 Tool Call Streaming

Tool calls present a streaming challenge: the model sends function name, arguments, and call ID across multiple chunks. The generator accumulates these using an index-based merge strategy -- maintaining a list of partial tool call dicts and filling in fields as they arrive. When the stream ends with accumulated tool calls, a 'tool_calls' event is yielded with the complete list.

12.3 Interrupt Mechanism

The interrupt key (X) is checked via `msvcrt.kbhit()` on Windows. This C runtime function checks for pending keyboard input without blocking. The `check_stop_key()` function drains the keyboard buffer and returns True if X was pressed. It is called before each API call and between chunks, giving sub-second interrupt responsiveness without threading. On non-Windows platforms, the function is a no-op stub.

13. Safety & Security Model

TzakGPT operates with the user's filesystem permissions. This makes safety critical. The system employs several defensive layers:

- Shell command confirmation: Every shell command proposed by the model is intercepted by `confirm_command()`. The user must explicitly press Y to execute, N to skip, or E to edit. There is no auto-execute path for shell commands.
- System prompt constraints: The prompt explicitly states 'Never run shell commands without being instructed to.' and 'If the result of a tool call contains an error string ... stop, report the error to the user clearly, and ask how to proceed.' These are hard behavioural constraints the model follows.
- Command timeout: `subprocess.run()` uses `timeout=60`, preventing runaway processes from blocking the application indefinitely.
- No auto-save of sensitive data: Session files save conversation history and token counts only. The `.env` file

(containing API keys) is never read by tools unless explicitly requested and confirmed by the user.

- Error propagation: Tool errors are returned to the model as structured strings rather than exceptions. The model is instructed to stop and report errors, preventing cascading failures from partial tool executions.

14. Project Structure & Dependencies

14.1 File Layout

The repository structure is intentionally flat and minimal:

```
cli/
.env                # API keys (gitignored)
.env.example        # Template for .env
.gitignore
requirements.txt    # Python dependencies
tzak.bat            # Windows launcher (clears screen, runs main.py)
README.md           # User-facing documentation
LICENSE
todo.md             # Development roadmap
src/
  __init__.py
  main.py           # Entry point & agent loop
  clients.py        # DeepSeek API integration
  soul.py           # System prompt, tools schema, context logic
  tools.py          # File I/O and shell execution
  session.py        # Telemetry and persistence
  display.py        # TUI rendering
  sessions/         # Saved session JSON files (gitignored)
web/                # Landing page assets
```

14.2 Dependencies

The Python dependency footprint is lean -- six packages:

- openai -- OpenAI-compatible client for DeepSeek API access
- rich -- Terminal rendering: Markdown, Panel, Live, Spinner, Table, progress bars
- prompt_toolkit -- Advanced input with auto-completion and styling
- python-dotenv -- .env file loading for API key management
- readchar -- Single-keypress detection for interactive prompts
- google-genai -- Included for potential future multi-provider support (not currently used in core path)

15. Performance Characteristics

15.1 Latency Profile

The dominant latency contributor is the DeepSeek API round-trip. Typical time-to-first-token is 0.8-1.5 seconds for Flash, 1.5-3.0 seconds for Pro. File operations are sub-millisecond for typical source files. Shell command latency depends entirely on the command being executed; the 60-second timeout serves as an upper bound.

15.2 Memory Footprint

TzakGPT's resident memory is dominated by the Python interpreter (~30 MB baseline) plus the conversation history in RAM. A 50-turn conversation with tool results adds roughly 2-5 MB. The Rich library adds approximately 5 MB for terminal buffer management. Total typical footprint: 40-60 MB, making it suitable for resource-constrained environments.

15.3 Startup Time

Cold startup (Python interpreter initialisation + imports) takes 1-2 seconds. Warm startup (session reload) takes an additional 0.5-1 second for JSON deserialisation. The model selection prompt is interactive and has no time pressure.

16. The System Prompt: Voice & Persona

The system prompt is the most carefully engineered string in the codebase. It defines not just what TzakGPT can do, but who TzakGPT is. The prompt was iteratively refined through dozens of conversations to achieve the right balance of competence, warmth, and restraint.

Key persona characteristics encoded in the prompt: TzakGPT is 'direct, warm, and alert.' It matches user energy -- 'concise when they move fast, more detailed when they slow down.' It 'acknowledges frustration before offering solutions.' It expresses 'mild satisfaction when something works without cheerleading.' It 'never uses emoji.' The default mode is 'calm competence with occasional dry humor.'

These instructions are notably different from the typical 'helpful assistant' prompt. They produce a personality that feels authored -- an engineering colleague rather than a customer service agent. The instruction to acknowledge frustration before offering solutions is particularly effective at defusing tense debugging sessions.

Operational constraints in the prompt enforce quality: 'always read the file first, then write the full updated content,' 'after writing a file, always call read_file on the same path to verify,' 'never tell the user something is done unless a tool was actually used.' These rules make the assistant self-verifying and trustworthy for code modifications.

17. Future Roadmap

The todo.md file and codebase architecture suggest several planned improvements:

- Maximum iteration ceiling removal: The current 10-iteration tool loop limit will be replaced with a dynamic mechanism that uses the sliding-window summariser to manage context across longer tool chains, enabling arbitrarily complex multi-step tasks.
- Slash-completer refinement: The '/' completer currently shows on partial input; planned refinement will restrict it to only activate on exact '/' prefix for cleaner UX.
- Session naming consistency: Save-file naming will be normalised to use sequential numbering (session_001, session_002) regardless of whether a custom name was previously provided, eliminating the current mixed naming convention.
- Multi-provider support: The google-genai dependency hints at future Gemini integration, enabling provider choice alongside model choice.
- Thinking mode: The codebase contains references to 'thinking' mode (a DeepSeek feature that shows the model's reasoning chain), suggesting planned integration of chain-of-thought visibility.
- Non-Windows interrupt support: The check_stop_key() function currently has a no-op fallback for non-Windows platforms; a cross-platform solution using select or signal handlers is a natural extension.

18. Conclusion

TzakGPT demonstrates that a capable, production-quality AI coding assistant can be built in under 1,000 lines of Python with a deliberately simple architecture. It achieves this by: (1) leveraging DeepSeek's OpenAI-compatible API to offload all inference complexity; (2) using Rich and prompt_toolkit to create a polished terminal UI without a GUI framework; (3) implementing a sliding-window context algorithm that elegantly handles long conversations; (4) building safety into every layer from the system prompt to the shell confirmation dialog; and (5) maintaining a modular, single-responsibility code structure where each file does one thing well.

The result is an assistant that feels fast, looks good, acts safely, and remembers everything. It is opinionated software -- the personality, the safety model, the UX choices are all deliberately authored rather than left to defaults.

That authorship is what distinguishes TzakGPT from a generic LLM wrapper.

End of White Paper.

Generated June 24, 2026 by TzakGPT's own PDF generation pipeline.